In *Chapter 2*, we generated and stored the synthetic transaction data in JSON files. Now, we generate similar data and read that data into a Delta table. First, notice the updates to the data generation notebook, `CH3-01-Generating Records and Schema Change`. Updates include a product string column, which will demonstrate schema evolution, as mentioned previously in the section on *Schema Evolution*. The `Product_vars` dictionary has been updated to include new products A, B, and C. The new products are then added to the transaction record generation mid-stream. In order to run the second notebook, you need to have this data generation notebook running first. Notice that we provide a list of possible products to the `writeJsonFile`. The result is an additional field, `Product`, with a product string added to the record (`Figure 3.4`).

```
import time

Products = ["None"]
t=1
total = 100
while(t<total):
  writeJsonFile(destination_path,Products=Products)
  t = t+1
  if not (t % 4):
    print(f"t={t}")
  time.sleep(sleepIntervalSeconds)
  if (t > total/4):
    Products = ["A","B","C"]
```

Figure 3.4 – Updated method for generating data with or without a product string

As a result of these changes, you now have a stream of data that adds an additional column midstream. You can start the Auto Loader notebook to see the schema evolution in action. The stream stops when it detects the extra column, providing an exception: `[UNKNOWN_FIELD_EXCEPTION.NEW_FIELDS_IN_RECORD_WITH_FILE_PATH]`.

Don't worry; upon restart, the schema evolution takes over. Run the cell with the stream in it again to restart. Restarts can be handled in workflows for production. You will see that in *Chapter 7*.

Let's walk through two options for reading and writing the data stream to a Delta table. Both options use Auto Loader to ingest. The files are then handled and written out by either Spark Structured Streaming or Delta Live Tables. There are two ingestion notebooks for this section. We start with `CH3-02-Auto Loader and Schema Evolution`.